

Cycles des vies des entités

DIEDLER Baptiste

10 juillet 2024

Objectif : L'objectif de ce stage est de formaliser les notions de vie et de cycle de vie d'une entité. Dans le but de pouvoir décrire toutes les évolutions possibles dans le temps de l'instance d'une base de données relationnelle sous l'effet d'un ensemble fini fixé de programmes. Soit pouvoir décrire le comportement d'une entité de sa création à sa destruction.

Table des matières

1	Rapport séance	4
1.1	31/05/2024	4
1.2	07/06/2024	5
1.3	12/06/2024	6
1.4	13/06/2024	6
1.5	14/06/2024	7
1.6	19/06/2024	7
1.7	26/06/2024	8
1.8	28/06/2024	8
1.9	02/07/2024	9
1.10	03/07/2024	10
1.11	08/07/2024	10
1.12	09/07/2024	11
1.13	10/07/2024	11
2	Application de base de données	12
3	Entité et cycle de vie	12
3.1		12
3.2	Entité et vie d'entité	12
3.2.1	Entité	13
3.2.2	Vie d'entité	13
3.2.3	Trace d'une vie d'entité	14
3.3	Cycle de vie	14
3.3.1	Syntaxe	14
3.3.2	Sémantique	14
3.4	Description du langage	16
3.4.1	Première grammaire	16
3.4.2	Vies d'entités et cycles de vies	18
3.4.3	Algorithmes	20
3.5	Structure d'une vie d'application	22
3.5.1	Combinaisons des vies d'entités et du schéma	22
3.5.2	Indépendance des vies d'entités	23
3.5.3	Dépendance des vies d'entités	23
3.5.4	Strates	24

4	Exemple achat d'articles	24
4.1	représentation de la base de données	24
4.2	exemples de programmes	25
4.3	exemple de cycle de vie d'entité	27
4.4	exemple de strates	28
4.4.1	exemple :	29
5	Démonstrations des observations	30
5.0.1	observation 3.5 :	30
5.0.2	conséquence observation 3.5 bis :	30
5.0.3	observation 3.6 :	30
5.0.4	cobobservation 3.6 bis :	31
5.0.5	observation 3.8 :	31
5.0.6	observation 3.17 :	31
5.0.7	observation 3.17 bis :	31
5.0.8	observation 3.20 :	32
5.0.9	observation 3.21 :	32
5.0.10	observation 3.22 :	32
5.0.11	observation 3.24 :	32
5.0.12	observation 3.25 :	33
5.0.13	observation 3.27 :	33

List of Algorithms

1	Successeurs. Les successeurs de q par $p.name$	17
2	Testcond. Savoir si q respecte $\langle COND \rangle$	18
3	CreationCEntite. Création du cycle de vie	21
4	CreationCApplication. Construction des cycles de vies	21

1 Rapport séance

1.1 31/05/2024

L'on peut changer d'état dans l'automate du cycle de vie d'une entité seulement si une action est possible sur notre entité. (Voir problème car cela dépend de l'état des tables autour) La vie d'une entité est une séquence d'état de l'entité où deux états consécutifs sont différents. La présence d'un arc signifie qu'il existe un moment où l'entité va appliquer cette action. Comment représenter une entité ? Une entité a-t-elle forcément une clef primaire. Peut-on construire le cycle de vie de Client dans SdV avec un seul état. Comment peut-on savoir les appels qui sont applicables à un instant t . Car un appel applicable doit peut-être attendre une configuration spéciale de la base de données. Et donc ne l'a pas encore. Donc la vie d'une entité est une séquence d'état changeant de celle-ci. On représente les états de notre automate du cycle de vie par des ensembles finis de programme possible à ce moment là.

On ne peut pas définir la vie de cette façon avec une représentation si faible du cycle de vie car on peut changer d'état dans le cycle de vie si une nouvelle action est possible. Mais sans pour autant changer les valeurs des attributs de l'entité.

On a donc pas de sommet auto-bouclé dans l'automate de cycle de vie.

Un automate de cycle de vie d'une entité est le même que pour toutes les entités de la même table. Problème si une action dépend de l'id de l'entité comme on fait une action seulement si l'id est inf à 10. Pour représenter une table on peut la voir comme une liste de : objet, relation (Commande), valeur (saison), autre ? Toutes les tables ont un cycle de vie ? Ainsi, un cycle de vie $\rightarrow$ une table donnée. Peut-on voir des similitudes entre différentes tables en fonction de leur cycle de vie. Dans l'automate nous avons obligatoirement deux états. L'état initial de création et l'état final de destruction. Et il nous faut au moins un autre état. Pour montrer que l'entité existe. Un état est bloqué, si l'ensemble des traitements applicables dessus est vide.

1. description cycle de vie d'entité.
2. description entité
3. description vie d'une entité

1.2 07/06/2024

Attention aux définitions qui n'en sont pas. Et aux remarques qui n'en sont pas non plus. Propriété : Caractériser toutes les vies de base de données est une union «disjointe» des vies d'entité. Donc l est une vie si et seulement si elle est représenté sous la forme d'une union de vies d'entité. Etude des propriétés locaux des cycles de vie (convergence). Voir le lien entre les entités plus tard. Mais si on ne manipule pas de contraintes, on ne prend pas en compte les liens. Faire une étude plus approfondit des strats. Mais on a un cas bizarre. On veut étudier Client et Village. Mais ils sont dépendant. Voir les reformulations de certaines phrases. Peut-être qu'il y a des cycles de vies qui ne dépendent de rien. On a donc peut-être une hiérarchie dans les cycles de vie. Préciser Valeurs (e) c'est quoi le produit cartésien de quels ensembles. La trace c'est une séquence de programme tel qu'il existe des paramètres à chaque programme qui génèrent la vie. Nous pouvons voir la trace comme le représentant d'une classe d'équivalence au sein des vies d'entité. Changer pour le moment la notation de trace par un autre nom ou par générateur. À un moment je dis que la vie des entités dépendent de la vie d'une base de données. Mais c'est peut-être l'inverse. On peut remplacer instancier par démarrer. Dans une table on peut avoir deux cycles de vie disjoints si deux tables sont mis au même endroit. Donc la remarque 1 est une erreur. On a que un cycle de vie est un automate d'état fini dont les états sont... et les traitements sont... Le langage reconnu par l'automate est $L(A) = \dots$ La trace est un mot du langage. Pour faire l'algorithme il faut prendre en input une application de base de données. Et qui renvoie un ensemble de cycles de vie d'entité. Défiir cycle de vie d'une base de données. Montrer la propriété de C d'une application. Pour tous les vies dans une application et pour toutes les vies d'entité dans la vie la vie d'entité satisfait l'un des cycles de vie du cycle de vie de l'application. Ne pas oublier de faire des exemples. Pousser les algorithmes jusqu'au bou. Et ne pas seulement le survoler.

1. description C automate états fini. / états ? / transitions ?
2. Parler du langage qu'il représente.
3. Parler de la vie satidfait un C si la trace appartient au langage.
4. Parler du cycle de vie d'une application
5. Parler de l'algo qui prend en entrée une application et qui renvoie le cycle de vie de l'application.
6. Parler d'une propriété

7. EXEMPLES

1.3 12/06/2024

Attention à la formulation de certaines phrases. Avoir de la rigueur dans les notations. Exemple ne pas utiliser des indices avec les variables pour les différencier. Seulement pour parler d'une position dans une liste. Quand viendra l'ajout des contraintes, on pourra avoir des ajouts d'états. Exemple avec Client de SdV qui peut avoir un état qui signifie que l'avoir ne peut plus changer. La présence de $\top$ est une bonne idée mais doit seulement être présent dans la vie et pas dans le cycle de vie. Passer le S^n en 2^S pour parler des sous parties d'un ensemble. Revoir les transitions car on ne veut pas les faire dépendre des paramètres. Ajouter $\top$ et $\perp$ dans les sommets. Si on a un exemple il faut noter la raison pour laquelle on a fait cet exemple. Dans des définitions, il ne faut pas oublier de parler de la vie d'une application de base de données. Hypothèse, une entité est présente une seule et unique fois. Dans l'algo, il ne faut pas oublier dans les paramètres A. Pour savoir si un attribut prend ses valeurs dans un ensemble de taille finie, on peut comparer si c'est un param ou une constante. Ne pas hésiter à faire des cas plus simples. Par exemple, on peut demander à l'utilisateur de transmettre l'information de la taille des ensembles d'attributs. On peut aussi réduire le problème à un seul attribut variable. Voir si on n'a aucune information, on peut le voir comme un attribut dans un ensemble infini. **Si on se place dans un contexte possible, il n'existe pas d'attribut qui peut prendre ses valeurs dans un ensemble infini. Car la mémoire est un espace fini.**

1. mettre à jour le document avec des éléments cleans
2. ne plus faire dépendre les définitions des dimensions pour le cycle de vie.
3. ne plus faire dépendre les transitions des paramètres
4. étudier sur des cas plus simple
5. ne pas oublier de dire pourquoi on est sur des cas plus simple.

1.4 13/06/2024

Poursuivre la notion des nouveaux états qui dépendent des programmes et non seulement de la taille de leur domaine de définition. Vérification de la

notion des transitions. Revoir les algorithmes au propre. Parler des propriétés qu'il y a autour comme l'union et des notions de propriétés locales.

1. mettre au propre la définition de cycle de vie
2. mettre au propre des algorithmes de construction du cycle de vie d'une application
3. apporter document logique
4. faire la démonstration de la première observation Descriptions of behavior of database applications
5. vérifier la propriété Soit C_A un cycle de vie d'une application, $\forall l \in \text{lives}(A), \forall v_e \in \text{Vies}_{entite}(l), v_e \models C_e \in C_A$

1.5 14/06/2024

Nous avons étudié comment faire la première démonstration de l'observation nupéro 1 de Descriptions of the behavior of database applications. Puis nous avons vu la structure de l'automate du cycle de vie d'une application. Ainsi que les algorithmes pour le construire.

1. faire la démonstration observation. Une tous les matins
2. faire la démonstration de la propriété sur les vies d'entité d'une vie d'application qui satisfassent un cycle de vie du cycle de vie de l'application
3. Etudier l'union des vies d'entité pour une vie d'application de base de données
4. Et continuer à faire les démonstrations d'observation.

1.6 19/06/2024

Nous avons étudié une démonstration dans les observations du document description of the behavior of database applications. Puis nous avons regardé la démonstration pour la propriété sur les vies et les cycles de vie.

1. revoir la démonstration mais dans l'idée elle est correcte
2. regarder si la démonstration fonctionne si la vie est infinie
3. regarder la propriété d'union disjointe
4. étude des propriétés locaux du cycle de vie

- (a) cycle
- (b) fini
- (c) infini

1.7 26/06/2024

On a vu les définitions pour les vies d'entités et des cycles de vie. Après peut-être qu'il faut changer le mot minimale par autre chose. On peut différencier plusieurs programmes. Des programmes quand les entités sont indépendantes. Donc on aura en paramètres l'id de l'entité ainsi que des valeurs supplémentaires. Et des programmes qui ne prennent pas en compte les dépendances des entités. Et peut on voir des dépendances entre les entités avec des programmes PL/SQL. Voir si l'on a plusieurs familles de vie d'entité. Comme des vies qui finissent forcément par mourir. Des vies qui finissent par des tout le temps les mêmes actions. Ou des vies dans boucles. Voir si la vie a un seul cycle de vie ou peut en avoir plusieurs. Voir si le cycle de vie dépend d'une table ou pas. Etudier les programmes pour voir lesquels sont utilisables.

1. définir la vie d'une entité
2. voir la propriété union
3. revoir le mot minimale -> représentant -> élément canonique
4. faire une étude des programmes
5. voir les différentes familles d'entité

1.8 28/06/2024

Nous nous sommes concentrés sur les définitions d'entité et de vie d'une entité. Nous avons parlé de primary key. Et de ce que ça représenté. Et voir si je ne pouvais pas généraliser la notion de clef primaire avec seulement un entier unique à plus générale. Comme un couple de clefs étrangères. Peut-être que modifier une clef primaire fait mourir une entité et en créer une nouvelle. Les instances d'application de base de données sont tirées de vies d'application de base de données. On est pas obligé de dire que quand une vie d'entité appartient à une vie d'application de db elle n'a les mêmes paramètres et les mêmes programmes que la vie de l'application. Une vie peut avoir plusieurs traces. Mais quand elle appartient à une vie d'application elle en a une seule.

Regarder s'il y a que des vies d'entités dans la vie d'une application. PS si on montre que la vie d'une application est une union de vies d'entités on a gagné.

1. définition entité
2. généraliser la clef primaire
3. expliquer le changement de clef primaire -> modifier un peu l'algorithme
4. voir si les instances d'une base de données sont tirées d'une vie de base de données -> Le problème est que l'on ne sait pas à quelle vie de db elle appartient. Si même il y en a une. Pas encore d'exemple.
5. revoir v dans l . -> ne plus avoir que les programmes sont les mêmes. On utilise un appel pour passer d'une instance à une autre et si appartient on prend les appels de la vie de db.
6. une vie a plusieurs traces et quand elle appartient à l elle en a une seule

1.9 02/07/2024

intuition du début de séance :

1. une vie est des entités qui respectent un cycle de vie
2. les entités ne sont pas forcément indépendantes
3. peut-être avons nous autre chose que des entités. Comme des liens. Voir [LWAJ8]
4. 1+2+3 caractérise une vie

Pour le point deux on peut (pour avoir seulement des entités) supposer que toute relation a sa colonne clef primaire. C'est à dire tout schéma est équivalent à un schéma dans lequel on ajoute une colonne clef primaire. A préciser et illustrer.

Nous avons un problème. On ne sait pas comment passer d'une vie d'entité à une vie d'application.

1. si $v_{e_0} \in Strate_1$ alors elle peut être créée à partir de $I_0 = \emptyset$.
2. Pour les autres cas nous verrons plus tard

Nous avons deux possibilité pour définir une vie d'entité

1. appartient à une vie application
2. n'appartient pas à une vie application (il en existe une)

C'est pourquoi nous avons décidé de prendre la deuxième définition.

Maintenant il nous faut parler de :

1. algo : app $A \rightarrow C$. Algo de construction des cycles de vie
2. propriété : $v_e \in l \in \text{lives}(A) \Rightarrow v_e \models C \in C(A)$. Est ce que la réciproque est vraie ?
3. propriété ultime : $\forall l \in S$ sans application est un conglomerat de vies d'entités.

remarque : Si les entités sont indépendantes alors la propriété ultime est triviale. Sinon elle est moins triviale, même pour le cas simple des strates qui semble être un cas de dépendance.

Définition d'indépendance : v_e est indépendante de $v_{e'}$ dans l si il existe l' avec v_e mais pas $v_{e'}$ ni toutes vies isomorphes à $v_{e'}$.

Question : Actuellement la définition de strat est sur les programmes. Pouvons nous définir les strats sur les entités ? Si oui peut-on les trouver grâce aux algo sur les cycles de vie ?

Conjecture : $\forall l$ peut-on ajouter n'importe quelle vie d'entité n'importe où dans l ? Y a-t-il un algo pour le faire ?

1.10 03/07/2024

Est-ce gênant qu'une entité puisse ressusciter ? Si c'est gênant on interdit les applications où on retrouve deux fois la même entité. On va nommer v_e une vie d'entité de e . Mieux définir la vie minimal de e .

1.11 08/07/2024

Nous avons vu la construction d'un cycle de vie et nous parlerons de l'algorithme la prochaine fois. Puis nous nous sommes rendu compte que la propriété ultime n'est pas une propriété mais une remarque car c'est dans la définition de la vie de l'application. Le travail sur les combinaisons est une partie sur la dépendance.

1. Est-ce qu'une vie d'entité est indépendante ?
2. Est-ce que chacune des vies satisfont un cycle de vie ?

1.12 09/07/2024

Explication de la définition du début d'une vie d'entité. Puis explication de la démonstration sur les cycles de vies. Avec lecture des algorithmes et de la grammaire. La conclusion est qu'il faut revoir des éléments inutilisés de la grammaire et réécrire les algorithmes en fonction de cette nouvelle grammaire.

1. refaire un point sur des définitions comme pour le début d'une vie ou encore la trace.
2. réécrire la grammaire
3. réécrire les algorithmes
4. Revoir la preuve en fonction des nouvelles notions.

1.13 10/07/2024

Lecture de l'assemblage du document. Puis de la démonstration sur les cycles de vies.

1. Faire les modifications expliquées dans la réunions
2. Relire l'hérédité puis la modifier en fonction de l'Initialisation.
3. Modifier les états avec seulement $q_1, \dots, q_n$
4. Refaire le document avec tout dedans.
5. Bien écrire tout ce qui se trouve dans le document.

Nous prendrons comme exemple d'application de base de données Séjours de Vacances. Avec comme relations Clients, Villages et Séjours. Et nous avons une dizaine de programmes. Comme par exemple le 1 qui ajoute un client et le deuxième qui achète un village par un client et ajoute un séjour dans la table Séjours.

Puis par la suite nous définissons une nouvelle application de base de données sur l'achat d'article par un devis.

2 Application de base de données

Soit A une application de base de données. Alors $A = (S, P)$ avec S le schéma de la base de données. Qui est l'ensemble fini des relations dans A . Et P l'ensemble fini des programmes exécutables dans la base de données respectant les règles de la grammaire décrite plus loin.

définition : Soit S un schéma d'une base de données. Alors $\forall R \in S$ R possède une colonne de clefs primaires.

remarque : Tous les attributs de la colonne des clefs primaires sont uniques et ne peuvent pas être modifiés.

3 Entité et cycle de vie

Dans la suite du document on note A une application de base de données.

3.1

3.2 Entité et vie d'entité

Soit S , $\forall R \in S$ a une colonne clef primaire sur S . Nous posons L un comportement de S définit dans descriptions of the behavior of database applications.

3.2.1 Entité

définition : Soit $l \in L$, une *entité* e dans l est la valeur de la colonne clef primaire d'une ligne d'une table R d'une instance I de l .
On dit alors que e est une entité de R et que sa *valeur* dans I est la ligne de $I(R)$ où e figure comme valeur de la colonne clef primaire de R .

exemple : (1,Riton,23,2000) est une valeur de l'entité 1 de la table *Client* de Séjours de Vacances.

3.2.2 Vie d'entité

définition : Soient $l \in L$, $e \in I$ de l . Alors il existe j minimal et k maximal et $I = I_n$ et $j \leq n \leq k$ où e apparait dans $I_j \dots I_k$ et $e \notin I_{j-1}$ et $e \notin I_{k+1}$.

Une *vie* de e dans l noté v_e est la séquence des $k - j + 1$ valeurs de e dans $I_j, \dots, I_k$ noté I^e .

Cette définition de vie d'entité nous permet d'étudier seulement celles qui sont de taille finie. Comment adapter la définition à des vies d'entités infinies ?

remarque : En particulier, comme $lives(A) \subseteq L$, on peut prendre $l \in lives(A)$. Car dans notre cas on étudie seulement les vies d'entités qui dépendent d'une application de base de données.

définition : Soit $v_e \in l \in L$, alors la *vie minimale* de v_e est la séquence v_e dans laquelle on a gardé une seule valeur dans toutes les sous séquences de valeurs identiques de v_e .

exemple : (1, Riton, 23, 2000); (1, Riton, 23, 1800); (1, Riton, 23, 1800);
(1, Riton, 23, 1700); (1, Riton, 23, 1454); (1, Riton, 23, 1454); (1, Riton, 23, 1454)
est une vie de l'entité 1 dans la table Client.

Sa vie minimale est (1, Riton, 23, 2000); (1, Riton, 23, 1800);
(1, Riton, 23, 1700); (1, Riton, 23, 1454).

définition : Soit $v_e \in l \in L$, alors un *début de vie* de v_e est un préfixe de la vie v_e .

On le note d_e . Et la séquence d'instances dans l de d_e est noté I^{d_e} avec $|I^{d_e}| \leq |I^e|$.

3.2.3 Trace d'une vie d'entité

définition : Soit $v_e \in l \in A$. On appelle *trace* de v_e la séquence de programme tel que pour chaque valeur, on prend le programme qui y a mené. Sauf si plusieurs valeurs successives égales, alors on ne considère, que le programme ayant mené à la première.

exemple : La trace de la vie de l'exemple au dessus est $[t1, t2, t2, t2]$

3.3 Cycle de vie

Soit A une application de base de données.

L'objectif du cycle de vie est de pouvoir représenter le comportement des vies des entités d'une relation de A . Ainsi on peut observer que le cycle de vie représente l'ensemble des vies que peuvent prendre les entités d'une table dans S .

définition : Le cycle de vie de A est l'ensemble constitué de cycle de vie de $R \in S$. On le note C_A .

3.3.1 Syntaxe

définition : Soit $R \in S$ cycle de vie des entités de R est un automate d'état fini. On le note C_R

remarque : Le langage reconnu par un cycle de vie d'entité est un langage régulier. Et plus précisément l'ensemble des traces de vies minimales des entités d'une table de S . On le note $\mathcal{L}(C)$.

3.3.2 Sémantique

Comme le cycle de vie est un automate d'état fini, alors on peut le représenter par un quintuplet $(\Sigma, Q, T, q_0, \delta)$.

Où Σ est l'alphabet qui est lu par l'automate. Q l'ensemble des états représentant la valeur d'une valeur d'entité après un appel de programme contenant p . T les états finals. Et q_0 l'état initial ainsi que δ une fonction de transition de $Q \times \Sigma$ vers Q .

Plus précisément :

- $\underline{\Sigma}$ est l'ensemble des noms de programmes dans A .

- $\underline{Q}$ est un ensemble indiquant les valeurs des entités. Et pour préciser l'origine de la valeur, l'ensemble sera composé de sommet étiqueté par $\top$, $\perp$ et de k -uplet avec k l'arité de la relation d'où viennent les entités représentés par le cycle de vie. Et chacun des éléments est étiqueté par le nom de la colonne de la table qui lui est associé. Si l'attribut de l'entité à la colonne col :
 - n'est pas modifié alors la valeur de l'élément du k -uplet étiqueté par col est **le nom de la colonne**.
 - est modifié par une constante. Alors la valeur de l'élément du k -uplet étiqueté par col est **la constante**.
 - est modifié par le i ème paramètre de p . Alors la valeur de l'élément du k -uplet étiqueté par col est le couple **(param,i)** où *param* est un mot clef indiquant que la modification a été provoquée par un paramètre.
 - est modifié par une expression de type «SELECT». Alors la valeur de l'élément du k -uplet étiqueté par col est **l'expression de type «SELECT»**.

$\top$ est un état qui exprime la que la vie existe. Et $\perp$ représente la valeur d'entité n'existant pas.

- $\underline{T}$ est égale à $\underline{Q}$.
- q_0 est $\top$.
- $\underline{\delta}$ est un ensemble de triplet dans $Q \times \Sigma \times Q$. Et que si $(q, t, q') \in \delta$. Alors il existe une instance I de A dans lequel il existe une entité e dont sa valeur correspond aux modifications de q . Et qu'il existe un appel avec le programme nommé t sur I nous fait passer dans I' . Et que la valeur de e dans I' correspond aux modifications de q' .

remarque : Nous pouvons caractériser les états comme des ensembles de programmes. Qui correspondent aux programmes entrant sur le sommet du graphe. Car deux sommets avec les mêmes ensembles de programmes entrant sont identiques.

remarque : Nous pouvons prendre la comme état les lignes où on a des insert, delete et update.

3.4 Description du langage

Dans cette partie nous voyons quel est le langage utilisé pour pouvoir construire le cycle de vie d'une entité. Dans le but de pouvoir construire la fonction *Successeurs* utilisée plus haut dans le document.

Pour cela nous nous sommes appuyé sur le langage décrit dans le rapport de Lyes. Mais dans un premier nous donnons un langage extrêmement plus simple pour pouvoir commencer.

3.4.1 Première grammaire

Le premier langage que l'on peut regarder est un langage qui décrit les programmes de l'application par la grammaire suivante :

```
<PROGRAMME> ::= <PROGRAMME>; <PROGRAMME> | <UP-  
DATE> | <INSERT>  
| <DELETE> | <SELECT>  
<SELECT> ::= SELECT  $att_1, \dots, att_n$  FROM <R> WHERE <COND>  
<UPDATE> ::= UPDATE <R> SET  $att_i = p$  WHERE <COND>  
<INSERT> ::= INSERT INTO <R> VALUES ( $p_1, \dots, p_n$ )  
<DELETE> ::= DELETE FROM <R> WHERE <COND>  
<COND> ::= <COND> AND <COND> |  $att = p$  |  $att \neq p$  |  $att < p$  |  $att$   
>  $p$   
<R> ::= l'ensemble des noms de relations dans  $S$ 
```

Où $p, p_1, \dots, p_n$ sont des constantes de l'ensemble des constantes *CONSTANTES* des programmes de l'application ou des noms de variables dans l'ensemble des noms de variables *VARIABLES* des programmes de l'application. Avec n le nombre de colonnes dans R .

Et $att_1, \dots, att_n$ les attributs de la table R .

Et <PROGRAMME> et l'axiome de la grammaire.

Nous faisons comme hypothèse que l'update ne peut pas être fait sur une clef primaire.

Dans la suite nous allons voir l'algorithme *Successeurs* pour la grammaire ci-dessus.

Nous prenons comme hypothèse que seul un insert peut créer une nouvelle entité. Car il est possible qu'un update modifie une clef primaire. Mais on ne

le prend pas en compte.

Algorithm 1: Successeurs. Les successeurs de q par $p.name$

Input: $prog$: un programme de A ; q : l'état d'où l'on vient; R : la relation étudiée par C

Output: v : le successeur de q par $prog$

```

1 if  $q = \top$  then
2   if  $prog = \text{'INSERT INTO } R \text{ SET VALUES } (p_1, \dots, p_n) \text{'}$  then
3      $v = (p_1, \dots, p_n)$ ;
4     for  $valeur_i \in v$  do
5       //  $valeur_i$  est la  $i$  ème valeur de  $v$ 
6       if  $valeur_i \in VARIABLES$  then
7          $valeur_i$  est remplacée par le nom de la colonne  $i$  de  $R$ ;
7 else
8   if  $prog = \text{'DELETE FROM } R \text{ WHERE } <COND> \text{'}$   $\wedge$ 
9      $TestCond(q, <COND>)$  then
10     $v = \perp$ ;
10  if  $prog = \text{'UPDATE } R \text{ SET } att_i = p \text{ WHERE } <COND> \text{'}$   $\wedge$ 
11     $TestCond(q, <COND>)$  then
12       $v = (att_1, \dots, att_n)$ ; // nom des colonnes de  $R$ 
13      if  $p \in CONSTANTES$  then
14         $v.att_i = p$ ;
14 return  $v$ 

```

exemple : On a le programme suivant de la grammaire avec *Client* une relation de Séjours de Vacances. T1 (Riton,23) = INSERT INTO *Client* VALUES (*nextVal*, *param*₁, *param*₂, 2000). Cela nous donne l'état suivant.

$(id_{client}, (param, 1), (param, 2), 2000)$

Car le première attribut est une clef primaire donc on le note par le nom de la colonne. Puis les deux attributs d'après viennent des paramètres donc on

Algorithm 2: Testcond. Savoir si q respecte $\langle \text{COND} \rangle$

Input: q : un état ; $\langle \text{COND} \rangle$: une condition de la grammaire

Output: *false* si q ne valide pas $\langle \text{COND} \rangle$ et *true* sinon

```
1 while  $cond \in \langle COND \rangle$  do
2   if  $p \in \text{CONSTANTES} \wedge q.att \in \text{CONSTANTES}$  then
3     if  $cond = (att = p) \wedge q.att \neq p$  then
4       return false
5     if  $cond = (att \neq p) \wedge q.att = p$  then
6       return false
7     if  $cond = (att < p) \wedge q.att \geq p$  then
8       return false
9     if  $cond = (att > p) \wedge q.att \leq p$  then
10      return false
11 return true
```

précise leur provenance. Puis le dernier attribut est une constante dans le programme on l'affiche dont avec sa valeur.

3.4.2 Vies d'entités et cycles de vies

Comme l'objectif est de pouvoir étudier les vies des entités, il est nécessaire de pouvoir faire le lien entre les vies et les cycles de vies. C'est pour cela que nous introduisons la notion de satisfiabilité.

définition : Soit v_e une vie de l'entité e de la table R dans S . Soit C_R le cycle de vie des entité de R . On dit que v_e *satisfait* C_R si $trace(v_e) \in \mathcal{L}(C_R)$.

propriété : Soit $v_e \in l \in lives(A) \Rightarrow v_e \models C \in C_A$.
(la réciproque n'est pas forcément vraie)

propriété : $\forall d_e \in v_e \in l \in lives(A) \Rightarrow trace(d_e) \in \mathcal{L}(C)$.
(la réciproque n'est pas forcément vraie)

Preuve :

Soit $v_e \in l \in lives(A)$. Prenons $C \in C_A$ le cycle de vie de la relation R où

évolue l'entité e . Montrons que $v_e \models C$.

Par récurrence sur la taille de d_e un début de vie de v_e .

Soit $P_n := \text{trace}(d_e) \in \mathcal{L}(C)$.

1. **Initialisation** ($|d_e| = 1$).

- La taille de la trace du début de vie d_e est de taille 1. Car sur une vie de taille 1 il n'y a pas de doublon.
- Par définition de la vie d'entité le programme p utilisé est un INSERT sur R .
- On commence par l'état $\top$.
- L'arc étiqueté par p sortant de $\top$ existe car p contient une ligne INSERT dans R . Et que dans l'algorithme pour construire le cycle de vie l'état $\top$ ne possède que des arcs sortants avec des INSERT. (Par définition du cycle de vie)
- Le sommet arrivant est final par définition du cycle de vie.

Donc P_1 est vraie.

2. **Hérédité** ($|d_e| = n + 1$).

Supposons P_n vraie.

- On a que $d_e = d'_e, v$. Avec $|d'_e| = n$.

(a) **cas 1 :**

Si v est égale à la dernière valeur de d'_e .

- Alors $\text{trace}(d'_e) = \text{trace}(d'_e, v)$ par définition de la trace.
- Donc comme $\text{trace}(d_e) = \text{trace}(d'_e) = \text{trace}(d'_e, v)$ et que par HR $\text{trace}(d'_e) \in \mathcal{L}(C)$
- $\text{trace}(d_e) \in \mathcal{L}(C)$

(b) **cas 2 :**

Si v est différent de la dernière valeur de d'_e .

- Alors $\text{trace}(d_e) = \text{trace}(d'_e).p$ avec p le programme nous faisant passer de la dernière valeur de d'_e à v .
- Par HR $\text{trace}(d'_e) \in \mathcal{L}(C)$. Donc le chemin suivi par la trace finit sur un état q .
- Il existe un arc étiqueté par p sortant de q . Car dans l'algorithme de construction du cycle de vie nous créons un arc sortant de q s'il est possible que le programme soit exécutable sachant que l'on est dans l'état q .

- Or il est exécutable sachant que l'on est dans l'état q car d_e est un début de vie qui respecte cela.
- Et l'état arrivant q' tel que $\delta(q, p) = q'$ est final par définition du cycle de vie.
- Alors $trace(d'_e).p \in \mathcal{L}(C)$
- $trace(d_e) \in \mathcal{L}(C)$
- Donc $trace(v_e) \in \mathcal{L}(C)$.

conjecture : Soit v_e une vie d'entité dans l une vie sur S . Alors si v_e satisfait un cycle de vie de A alors l est une vie de l'application.

remarque : Le cycle de vie est complet pour pour l'ensemble des vies d'une application. Donc $lives(A) \subseteq C_A$.

Mais elle n'est pas correcte. Car le cycle de vie reconnaît des traces de vies minimales. Et pas de simples vies d'entités.

3.4.3 Algorithmes

Dans cette partie nous allons voir des algorithmes permettant de construire le cycle de vie d'une application. Donc dans un premier temps nous verrons un algorithme pour construire le cycle de vie des entités d'une table $R \in S$. Puis après un algorithme pour construire le cycle de vie de A .

Cycle de vie d'entité L'algorithme qui suit calcule le cycle de vie des entités d'une table R donnée en paramètre.

Nous remarquons que l'algorithme revient à parcourir le graphe de l'automate en largeur. Mais il est aussi possible de le construire avec un parcours en profondeur ou avec de la programmation dynamique.

Dans cet algorithme nous appelons une fonction *Successeurs* qui renvoie les états vers lesquels l'état q possède un arc étiqueté par le nom du programme p . Cette fonction peut-être réalisé en faisant une étude du programme grâce à la grammaire décrite par Lyes dans son rapport. Il sera traité plus tard dans le document dans la partie description du langage.

Cycle de vie d'une application Par définition du cycle de vie d'une application on en tire l'algorithme suivant :

Algorithm 3: CreationCEntite. Création du cycle de vie

Input: A : application de base de données ; R : relation de S

Output: C_e : le cycle de vie des entités de R

```
1  $file = [\top]$ ;
2  $\delta = \emptyset$ ;
3  $\Sigma, F = \emptyset$ ;
4  $Q = \{\top\}$ ;
5  $s_0 = \top$ ;
6 while  $file \neq \emptyset$  do
7    $q = file.defile()$ ;
8   if  $q \neq \perp$  then
9     for  $p \in P$  do
10      for  $succ \in Successeurs(p, q, R)$  do
11         $\delta.add((q, p.name, succ))$ ;
12        if  $succ \notin Q$  then
13           $Q.add(succ)$ ;
14           $file.add(succ)$ ;
15       $\Sigma.add(p.name)$ ;
16    $F.add(q)$ ;
17  $A_{entite} = (\Sigma, S, s_0, \delta, F)$ ;
18 return  $A_{entite}$ 
```

Algorithm 4: CreationCAApplication. Construction des cycles de vies

Input: A : une application de base de données

Output: C_A : le cycle de vie de A

```
1  $C_A = \emptyset$ ;
2 for  $R \in S$  do
3    $C_A.add(CreationCEntite(A, R))$ ;
4 return  $C_A$ 
```

Non remarquons que *CreationCEntite* est l'algorithme au dessus pour construire le cycle de vie des entité d'une table de S .

3.5 Structure d'une vie d'application

Après avoir trouvé une structure permettant de décrire les vies d'entités. Maintenant voyons le lien entre les vies d'entités et les vies d'applications de base de données.

remarque : Une vie du schéma est un conglomerat de vies d'entités.

Par conglomerat nous entendons qu'à l'intérieur d'une vie du schéma nous avons seulement des vies d'entités. Car dans une instance de vie du schéma nous avons seulement des valeurs d'entités car c'est comme ça que nous avons construit notre schéma.

3.5.1 Combinaisons des vies d'entités et du schéma

Nous posons pour la suite S un schéma de base de données, L un comportement sur S et l une vie de L .

définition : Soient v_1, v_2 deux vies d'entités différentes. On a $I^1 \in l_1$ et $I^2 \in l_2$ les sous séquences des instances de S où se trouvent v_1 et v_2 . Soit I^3 une séquence d'une vie de S . Si I^3 a comme préfixe I^1 et suffixe I^2 ou inversement. Alors I^3 est une *combinaison* de v_1 et de v_2 .

exemple : Soit v_1, v_2 deux vies d'entités. Et $I^1 = I_1, \dots, I_n$ et $I^2 = I'_1, \dots, I'_m$. Alors $I^3 = I_1 \cap I'_1, \dots, I_n \cap I'_n, \dots, I'_m$ est une combinaison de v_1 et v_2 .

définition : Soient l une vie du schéma, $v_1 \in l_1$. Une *combinaison* de l et v_1 est une vie d'application l' où il existe $I' = I_m, \dots, I_n$ une sous séquence de l' tel que $I' = I^1$ et que il existe $I = I_p, \dots, I_q$ dans l où $I = I' \setminus I^1$.

Nous remarquons dans Séjour de Vacances que Client a besoin de Village pour évoluer. On a donc déjà un exemple de dépendance entre les entités de Client et de Village.

3.5.2 Indépendance des vies d'entités

Par la suite nous allons faire face à deux cas. Soit les entités sont indépendantes les unes des autres. Ou alors il y a au moins une entité qui dépend d'une autre.

Ainsi nous pouvons nous demander si les vies d'entités d'une vie d'un schéma sont indépendantes.

Nous posons pour la suite S un schéma de base de données, L un comportement sur S et l une vie de L .

propriété : Soient $v_1 \in l_1 \in L, v_2 \in l_2 \in L$ indépendantes l'une de l'autre. Alors pour toutes les combinaisons c de v_1 et v_2 , il existe l une vie de S tel que $c \in l$.

remarque : Si toutes les vies d'entités sur S sont indépendantes. Alors il existe une seule strate.

propriété : Toutes les entités pouvant commencer leur vie dans I_0 l'instance vide (toutes les relations sont vides) ne sont pas dépendantes de d'autre entité au départ.

3.5.3 Dépendance des vies d'entités

Soit S un schéma de base de données. L un comportement de S , $l \in L$ une vie sur S . Et V un ensemble de vies d'entités sur S

définition : Soient v_e et $v_{e'}$ deux vies d'entités appartenant à $l \in S$. On dit que e est indépendant de e' si et seulement si il existe $l' \in S$ tel que $v_e \in l$ et $v_{e'} \notin l$. Avec $v_{e''}$ toutes vies isomorphe à $v_{e'}$.

remarque : Toutes vies sur S sont représentables par des combinaisons de vies d'entités.

conjecture : Soit $l \in S$, alors nous pouvons ajouter n'importe quelle vie

d'entité v_e n'importe où dans l .

remarque : Trivial si les entités sont indépendantes. On peut prendre n'importe laquelle des combinaisons entre l et v_e .

3.5.4 Strates

Pour pouvoir parler de strate, il faut pouvoir définir différentes notions. Car nous remarquons la présence de strate par les programmes que l'on peut appliquer sur les entités d'une vie. Mais on a aussi des strates sur les vies d'entités et pas seulement pour les vies d'application.

remarque : On peut voir les strates comme des niveaux de dépendance entre les entités. Si strate 1 aucune dépendance. Si strate 2 dépend de la strate 1. Et la strate 3 dépend de la 2 mais par transitivité aussi de la 1.

conjecture : Le passage d'une strate par un appel incluant un INSERT à une autre se fait par l'ajout ou la suppression d'au moins une dépendance.

idée de la preuve :

Soit s_1 et s_2 deux strates. Si la taille de s_1 est plus grande que s_2 alors on a ajouté d'une dépendance. Car on a la possibilité de faire plus d'appel. Quand on a ajouté une entité. Les entités impliquées par cet appel dépendent de l'entité ajoutée. (resp pour diminué)

4 Exemple achat d'articles

Dans cette section nous retrouvons un exemple d'application de base de données. Qui représente l'achat d'article par un client.

4.1 représentation de la base de données

Nous y retrouvons S le schéma de l'application :

1. Client : $(id_{client}, nom, avoir)$

2. Devis : $(id_{devis}, id_{client}, statut)$
3. Commande : $(id_{commande}, id_{devis}, id_{article}, nombre)$
4. Article : $(id_{article}, nom, prix, nombre)$

Les programmes sont les suivants :

1. t1 : ajouter un client. Input le nom du client. Et l'avoir est initialisé à 2000.
2. t2 : créer un devis. Input id du client. Le statut est initialisé à «en cours».
3. t3 : ajouter des articles au devis. Input id du devis, id de l'article et le nombre que l'on ajoute. Le statut est changé pour être «en cours»
4. t4 : ajouter un article. Input nom de l'article, son prix, et son nombre.
5. t5 : vérifier un devis. Input id du devis. (Ce traitement vérifie que le devis est bien réalisable pour le client. Qu'il puisse le payer) Le statut du devis est modifier en «valide» si le client peut payer. Ou «invalidé» si le client ne peut pas.
6. t6 : annuler un devis. Input id du devis. Le statut devient «annulé».
7. t7 : payer un devis. Input id du devis. Fonctionne seulement si le statut est «valide». Le statut passe à «payé».
8. t8 : supprimer des devis. (Supprime les devis dont le statut est annulé)

Ainsi le statut d'un devis peut avoir plusieurs valeurs : en cours, valide, invalide, payé, annulé.

4.2 exemples de programmes

Dans cette section nous avons des exemples de programmes qui repecte la grammaire donnée plus haut.

traitement 1 de l'application. Où p1 est le paramètre :

```
INSERT INTO Client VALUES (next_val,p_1,2000);
```

traitement 2 de l'application. Où p1 est le paramètre :

```
INSERT INTO Devis VALUES (next_val,p_1,en_cours);
```

traitement 3 de l'application. Où p1, p2, p3 sont les paramètres :

```
INSERT INTO Commande VALUES (next_val,p_1,p_2,p_3);
```

traitement 4 de l'application. Où p1, p2, p3 sont les paramètres : (on pourrait avoir un meilleur algorithme en accumulant le nombre d'un article si on ajoute en sachant qu'il existe déjà)

```
INSERT INTO Article VALUES (next_val,p_1,p_2,p_3);
```

traitement 6 de l'application. Où p1 est le paramètre :

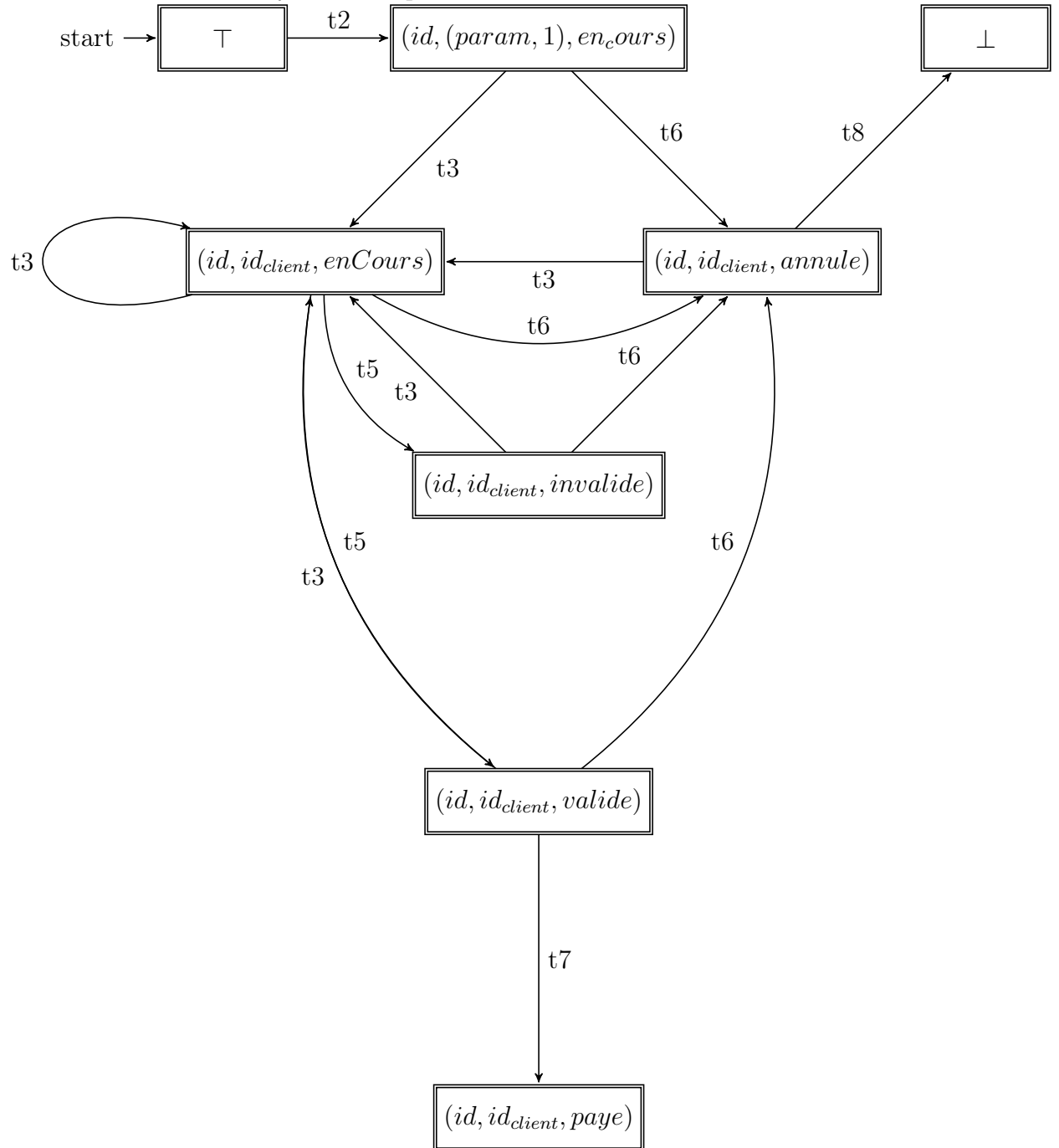
```
UPDATE Devis SET statut = 'annulé' WHERE id_devis = p_1;
```

traitement 8 de l'application :

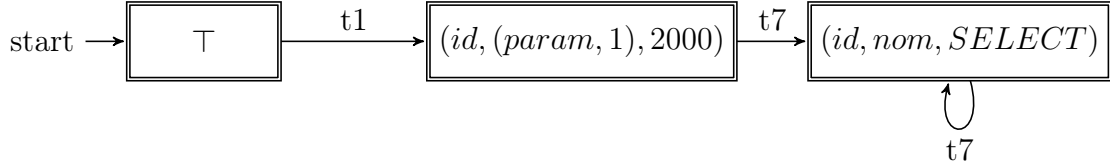
```
DELETE FROM Devis WHERE statut = 'annulé':
```

4.3 exemple de cycle de vie d'entité

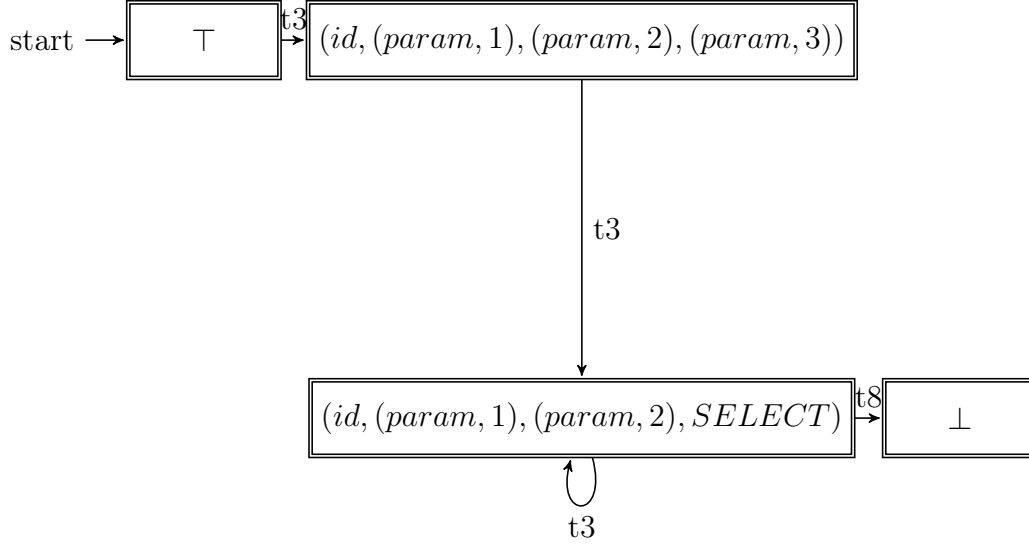
Voici l'automate du cycle de vie pour la table Devis.



Et maintenant l'automate du cycle de vie pour la table Client.



Et maintenant l'automate du cycle de vie pour la table Commande.



4.4 exemple de strates

Dans cette section nous avons un bon exemple de présence de strates. Nous avons pour la première strate $\{t1, t4\}$ car se sont les deux programmes qui modifient la base de données sans avoir besoin d'autres entités.

Puis nous avons la deuxième strate $\{t1, t4, t2\}$ car l'on peut toujours faire les actions de la strate 1 mais on peut aussi ajouter des devis.

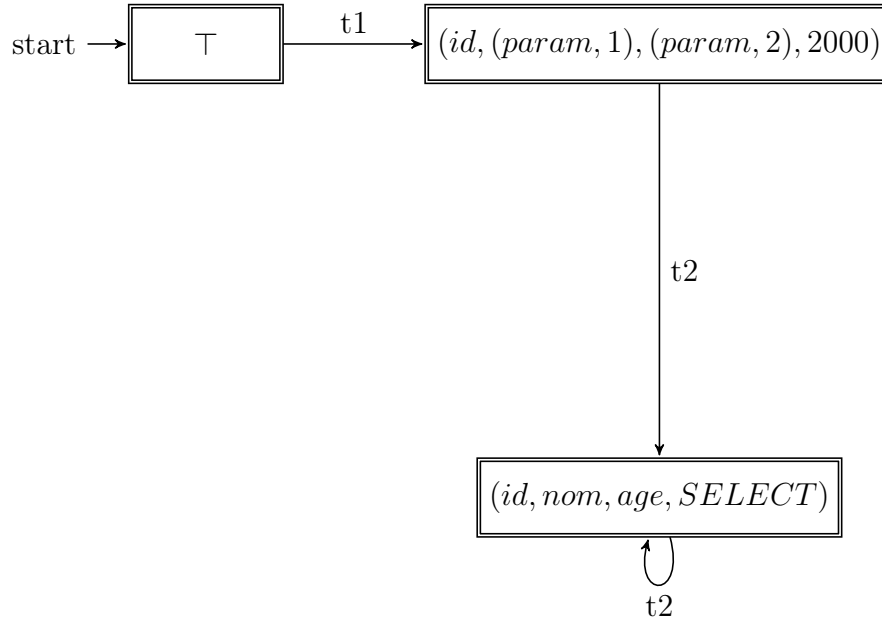
Et on a aussi la strate $\{t1, t4, t5, t6, t7, t8\}$. Elle arrive si et seulement si l'on a pas d'article dans la table article.

Et on a la dernière strate avec tous les programmes.

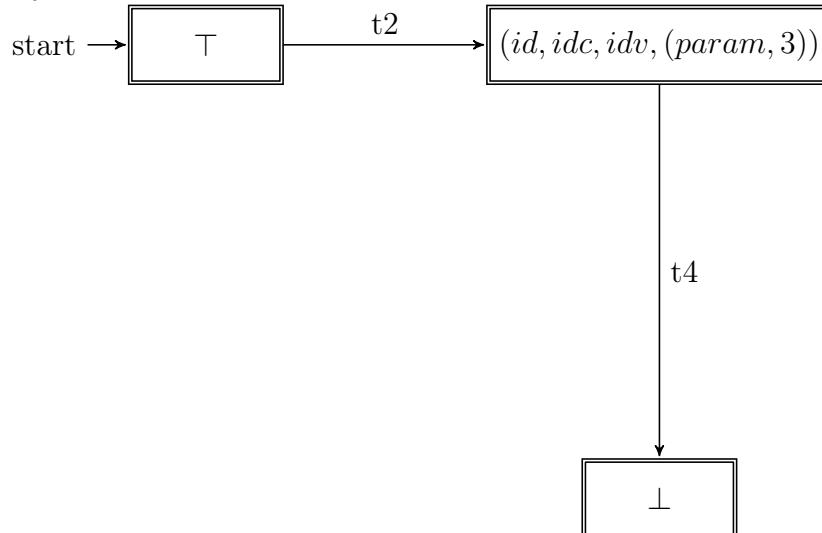
Dans cette application nous nous rendons bien compte qu'un devis dépend forcément des articles qui sont dans la table Articles.

4.4.1 exemple :

Pour cet exemple nous allons prendre le cycle de vie des entité de la table Client de Séjours de Vacances.



Puis nous allons prendre le cycle de vie des entité de la table Séjour de Vacances.



5 Démonstrations des observations

5.0.1 observation 3.5 :

Soit S un schéma et P un ensemble non vide de primitives ayant chacune des relations différentes. Avec $|P| \leq |S|$

Soit I une instance tel que $\forall p_i \in P$: (On note R_i la relation associée à p_i . R_i est associée à une unique primitive, soit $R_i^{\nearrow}, R_i^{\searrow}, R_i^{\rightarrow}$)

1. si $p_i = R_i^{\nearrow}$ alors $I(R_i) = \emptyset$
2. si $p_i = R_i^{\searrow}$ alors $I(R_i) = \{\text{k-uplet}\} \in \text{dom} \times \dots \times \text{dom}(\text{k fois})$
3. si $p_i = R_i^{\rightarrow}$ alors $I(R_i) = \emptyset$

Soit I' une instance tel que $\forall p_i \in P$: (On note R_i la relation associée à p_i . R_i est associée à une unique primitive, soit $R_i^{\nearrow}, R_i^{\searrow}, R_i^{\rightarrow}$)

1. si $p_i = R_i^{\nearrow}$ alors $I'(R_i) = \{\text{k-uplet}\} \in \text{dom} \times \dots \times \text{dom}(\text{k fois})$
2. si $p_i = R_i^{\searrow}$ alors $I'(R_i) = \emptyset$
3. si $p_i = R_i^{\rightarrow}$ alors $I'(R_i) = \emptyset$

Soit $p_i \in P$:

1. si $p_i = R_i^{\nearrow}$ alors $|\emptyset| < \{\text{n-uplet}\}$ on a donc $|I(R_i)| < |I'(R_i)|$
2. si $p_i = R_i^{\searrow}$ alors $\{\text{n-uplet}\} > |\emptyset|$ on a donc $|I(R_i)| > |I'(R_i)|$
3. si $p_i = R_i^{\rightarrow}$ alors $|\emptyset| = \emptyset$ on a donc $|I(R_i)| = |I'(R_i)|$

D'après la définition 3.3 la transition I, I' réalise P .

5.0.2 conséquence observation 3.5 bis :

Comme une formule est un ensemble de N primitives ayant chacune des relations différentes dans le schéma S . Avec $|S| = N$. Ainsi d'après la propriété au-dessus, il existe une transition qui réalise la formule.

5.0.3 observation 3.6 :

Soit S un schéma et I, I' une transition. Soit $R \in S$:

1. si $|I(R)| = |I'(R)|$ alors $I, I' \models R^{\rightarrow}$
2. si $|I(R)| < |I'(R)|$ alors $I, I' \models R^{\nearrow}$
3. si $|I(R)| > |I'(R)|$ alors $I, I' \models R^{\searrow}$

5.0.4 cobobservation 3.6 bis :

Comme chaque transition realise un ensemble de primitives ayant chacune des relations différentes, alors cet ensemble de primitives est une formule. Donc toutes transitions réalisent une unique formule.

5.0.5 observation 3.8 :

Soit S un schéma, D une description sur S et l une vie sur S . Nous avons que $l \models D$. D'après observation 3.6, $\forall I, I' \in l, \exists f \in D, I, I' \models f$.

Alors $I, I' \models f \in D$ et $I, I' \models f' \in D$. Montrons que $f = f'$

Soit $p \in f$: (R est la relation associée à p)

1. si $p = R^{\rightarrow}$ alors $|I(R)| = |I'(R)|$ donc $p \in f'$
2. si $p = R^{\nearrow}$ alors $|I(R)| < |I'(R)|$ donc $p \in f'$
3. si $p = R^{\searrow}$ alors $|I(R)| > |I'(R)|$ donc $p \in f'$

Donc $f \subseteq f'$.

Soit $p \in f'$: (R est la relation associée à p)

1. si $p = R^{\rightarrow}$ alors $|I(R)| = |I'(R)|$ donc $p \in f$
2. si $p = R^{\nearrow}$ alors $|I(R)| < |I'(R)|$ donc $p \in f$
3. si $p = R^{\searrow}$ alors $|I(R)| > |I'(R)|$ donc $p \in f$

Donc $f = f'$.

Ainsi $\forall I, I' \in l, \exists! f \in D, I, I' \models f$.

5.0.6 observation 3.17 :

Soit S un schéma, et L un ensemble de vie sur S . Nous notons D un description de L .

Soit $l \in L$ un vie de L . Alors D_l la description naturelle de l est un sous ensemble de D . Ainsi comme $D_l \subseteq D$ on a donc $lives(D_l) \subseteq lives(D)$ se qui nous donne $l \in lives(D_l) \subseteq lives(D)$. En conclusion $L \subseteq lives(D)$.

5.0.7 observation 3.17 bis :

Soit S un schéma et L un ensemble de vie. Montrons que D l'ensemble de toutes les formules satisfaites par les transitions dans L est minimale sachant que $L \subseteq lives(D_L)$. Si on prend l'ensemble $D' = D \setminus \{f\}$, avec $f \in D$ un formule. Alors $\exists I, I' \in l \in L, I, I' \models f \Rightarrow l \not\models D'$. Ainsi $L \not\subseteq lives(D')$.

Donc D est minimale.

Or la description naturelle de L est minimale. Donc D est la description naturelle de L .

5.0.8 observation 3.20 :

Soit S un schéma, et L un ensemble de vie non null sur S .

1. ($\Rightarrow$) On a donc $\exists D$ une description non vide tel que $\text{lives}(D) \subseteq L$. Comme $\text{lives}(D)$ est un sous ensemble de L et que $\text{lives}(D)$ est trivialement fermé, $\exists L' \subseteq L$ qui est fermé non vide. $L' = \text{lives}(D)$.
2. ($\Leftarrow$) On a donc $\exists L' \subseteq L$ avec L' qui est fermé non vide. Ainsi la description naturelle $D_{L'}$ est une description non vide. Et on a bien $\text{lives}(D_{L'}) \subseteq L$. Ainsi il existe une description D non vide tel que $\text{lives}(D) \subseteq L$. Cette description est $D_{L'}$.

5.0.9 observation 3.21 :

Soit S un schéma. Montrons qu'il y a un nombre fini d'ensembles de vies fermés. Soit L l'ensemble de toutes les closures. Alors $L = \bigcup_{c \in \text{Closures}} c$. Or $|\text{Closures}| \leq |D_n| \leq |D|$ avec D_n l'ensemble des descriptions naturelles et D l'ensemble des descriptions. Or $|D| \leq 3^{|S|}$.

5.0.10 observation 3.22 :

Soit S un schéma et L, L' deux ensembles de vies fermés sur S . Soit $l \in L \cup L'$.

1. Si $l \in L$ alors on a D_l la description naturelle de l . Et $\text{lives}(D_l) \subseteq L$ car L est fermé. Or $L \subseteq L \cup L'$ donc par transitivité de l'inclusion $\text{lives}(D_l) \subseteq L \cup L'$.
2. respectivement pour L' .

Ainsi $\forall l \in L \cup L', \text{lives}(D_l) \in L \cup L'$.

5.0.11 observation 3.24 :

Soit S un schéma.

1. ($\Rightarrow$). Soit L un ensemble de vie sur S . Et D une description sur S tel que $L \subseteq \text{lives}(D) \wedge \text{lives}(D) \subseteq L$. Alors $\text{lives}(D) = L$.

2. ($\Leftarrow$). Soit D une description sur S . Alors $lives(D) \subseteq L \wedge L \subseteq lives(D) \Rightarrow lives(D) = L$.

5.0.12 observation 3.25 :

Soit S un schéma et D une description sur S . Et L un ensemble de vie sur S . Montrons que si D est correcte et complète pour L alors $L = lives(D)$. Comme D est correcte pour L alors $lives(D) \subseteq L$. Et comme D est complète pour L alors $L \subseteq lives(D)$. Ainsi par double inclusion, $lives(D) = L$. Donc D est correcte et complète pour exactement $lives(D)$.

5.0.13 observation 3.27 :

Soit S un schéma. Et D une description sur S . Soit F une formule réalisable dans D . Alors $\exists l \in lives(D)$ tel que l réalise F . Or d'après observation 3.6 F